

Session 2 — Homework

JavaScript: data, loops and the page

Hand in before: the start of Session 3 **Time you need:** about 60–90 minutes for CORE

Objective

Practise the JavaScript we used today: arrays, objects, `for...of` loops, `if`, functions, template literals, and changing the page.

Before you start

1. Open the TaskFlow project you have from the end of today's session.
2. Check that the three filter buttons still work.
3. Work in `index.html`, `css/style.css` and `js/app.js`.

CORE — everybody does this

1. Add more tasks

Add these three tasks to the `tasks` array in `js/app.js`:

id	title	completed
7	Read about JavaScript objects	true
8	Help a teammate with CSS	true
9	Prepare questions for the next session	false

Then **do not change anything else** and open the page.

Check that:

- all nine tasks appear,
- the numbers become Total 9, Completed 4, Pending 5,
- all three filters still work.

Write down in one sentence *why* you did not have to change any other code. Bring that sentence to the next session.

2. Show a progress sentence

Under the task list, show a line of text like:

4 of 9 tasks completed

Do it like this:

1. In `index.html`, add an empty paragraph with the id `progressText`, after the task list.
2. In `css/style.css`, make it small, grey and centred.
3. In `js/app.js`:
 - get the element with `querySelector`,
 - write a function called `updateProgressText()`,
 - inside it, count the completed tasks with a `for...of` loop,
 - build the sentence with a template literal,
 - put it on the page with `.textContent`.
1. Call your function when the page starts.

Hint: the counting loop is almost the same as the one inside `updateStats()`.

Hint: a template literal uses back-ticks: `` ${completed} of ${tasks.length} tasks completed ``

CORE acceptance criteria

- ☐ Nine tasks appear in the list.
 - ☐ The cards show 9, 4 and 5.
 - ☐ A sentence under the list says `4 of 9 tasks completed`.
 - ☐ The sentence is built with a template literal, not by joining strings with `+`.
 - ☐ The number in the sentence is **counted by a loop**, not typed by hand.
 - ☐ If you add a tenth task, the sentence updates by itself.
 - ☐ No red errors in the console (F12).
-

STRETCH — if CORE was comfortable

3. Search by title

Add a search box above the task list. While the user types, show only the tasks whose title contains what was typed.

Requirements:

- an `<input type="text">` with an id, styled to fit the page,
- searching must **not** care about capital letters: typing `JAVA` must find *Practice JavaScript*,
- an empty search box shows everything again,
- the statistics do **not** change while searching.

Hint: listen for the `"input"` event, not `"click"`.

Hint: `.toLowerCase()` makes text lower case, and `.includes("word")` answers true or false.

Hint: store what the user typed in a variable, exactly like `currentFilter`, then call `renderTasks()`.

STRETCH acceptance criteria

- ☐ Typing filters the list immediately.
 - ☐ Upper and lower case both work.
 - ☐ Clearing the box shows all tasks again.
 - ☐ The three numbers never change while typing.
-

CHALLENGE — for students who want more

4. Search and filter at the same time

Make the search box and the status buttons work **together**.

Example: click **Pending**, then type `java`. You must see only tasks that are pending **and** contain "java".

Requirements:

- clicking a filter keeps the current search text,
- typing keeps the current filter,
- when nothing matches, show your "No tasks to show." message.

Hint: a task must pass **two** checks before you `push()` it.

Hint: it is easier to read if you build two `true / false` variables first, for example `matchesFilter` and `matchesSearch`, and then use `if (matchesFilter && matchesSearch)`.

CHALLENGE acceptance criteria

- ☐ Filter and search work together in both orders.
- ☐ Clearing the search keeps the selected filter.
- ☐ "No tasks to show." appears when there is no match.
- ☐ `getVisibleTasks()` is still one function that is easy to read.

How to hand in

1. Make sure there are no red errors in the console.
2. Keep your file structure exactly as it is:

```
index.html
css/style.css
js/app.js
```

1. Bring your project to the next session. Be ready to **explain one function line by line**.

Reminders

- Do not use `.filter()`, `.map()` or `.reduce()` yet. Practise the loop.
- Do not delete tasks from the array to make something work.
- `true` and `"true"` are not the same thing.
- If something is `null`, a `querySelector` did not find your element. Check the id in both files.